MEMORIA FINAL

Diseño y Programación Orientada a Objetos

Judith Caralt

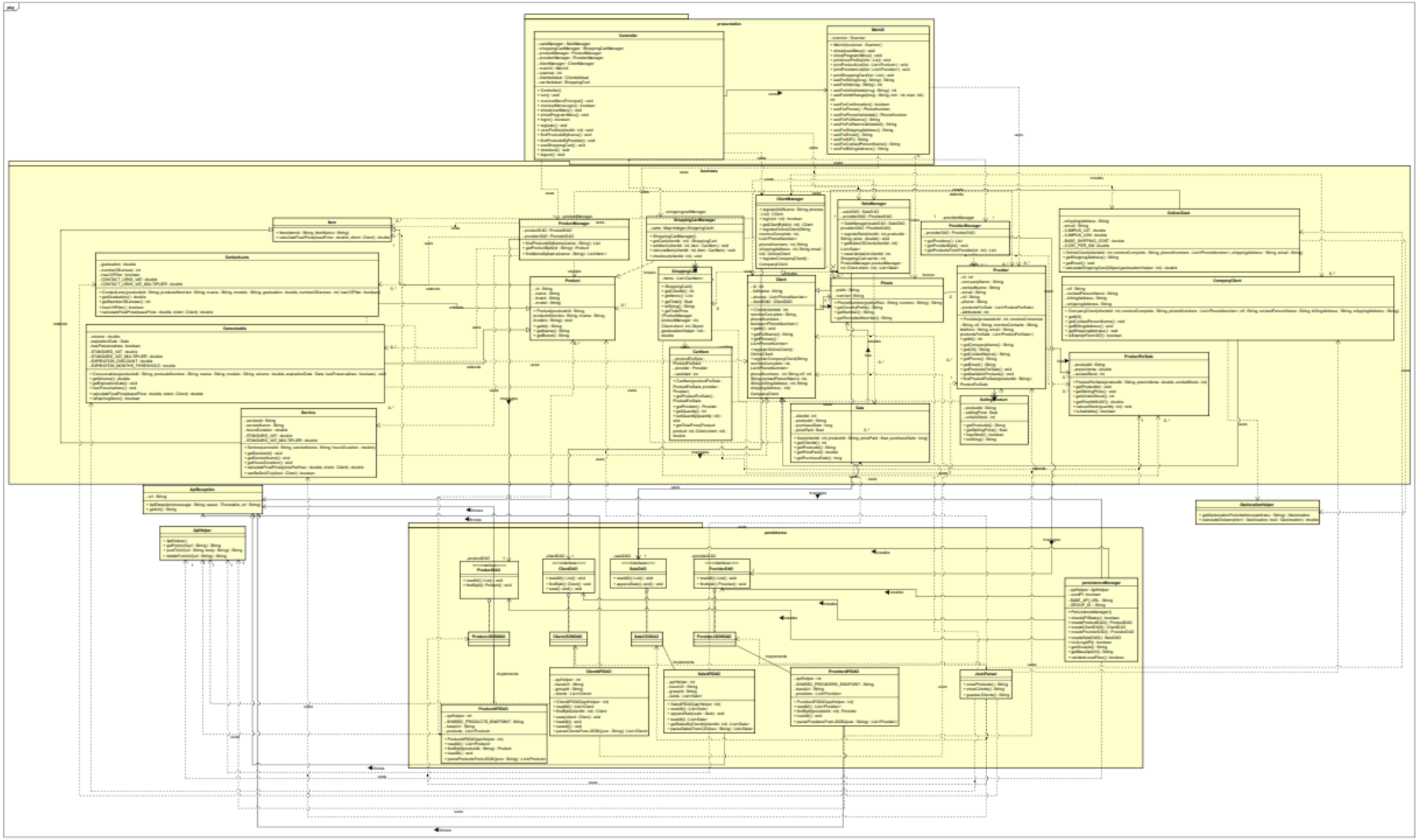
Aras Mora

ÍNDICE

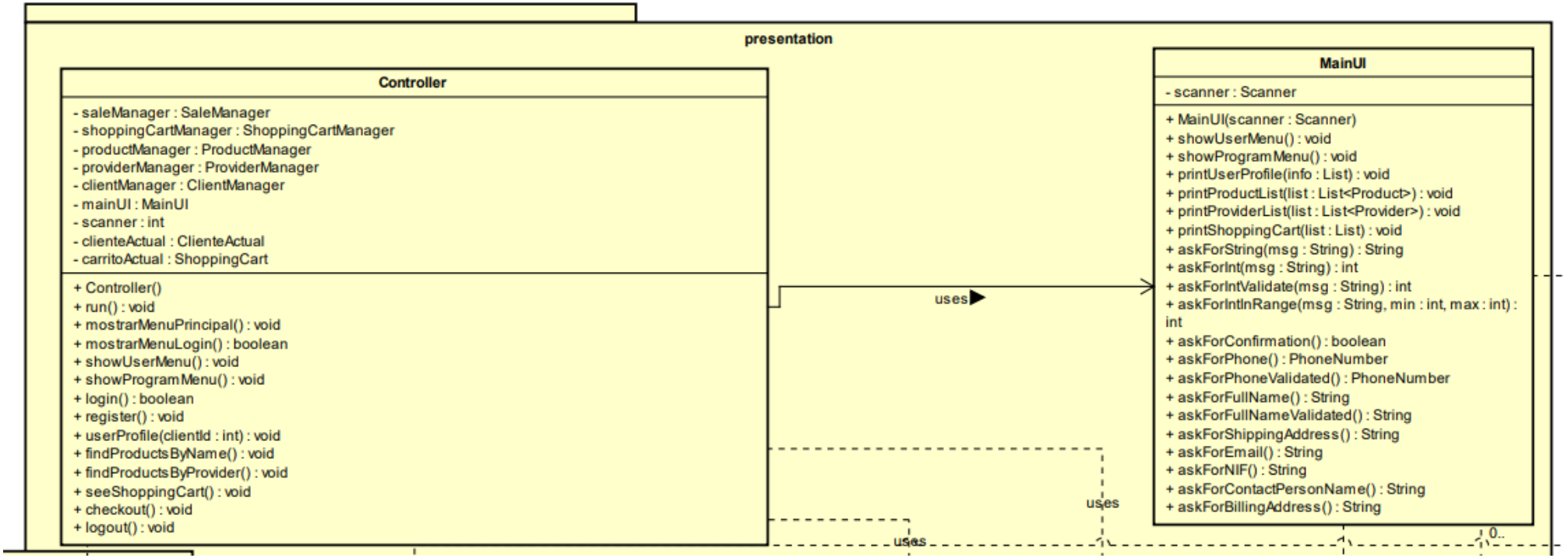
CONTEXTO 10
ARQUITECTURA DEL SOFTWARE 11
DECISIONES DEL DISEÑO 12
GESTIÓN DE DATOS 14
DOMINIO Y JERARQUIA DE ENTIDADES 15
FLUJOS DE CONTROL 16
DEDICACIÓN 17
CONCLUSIONES 18
BIBLIOGRAFÍA 19

DIAGRAMA DE CLASES

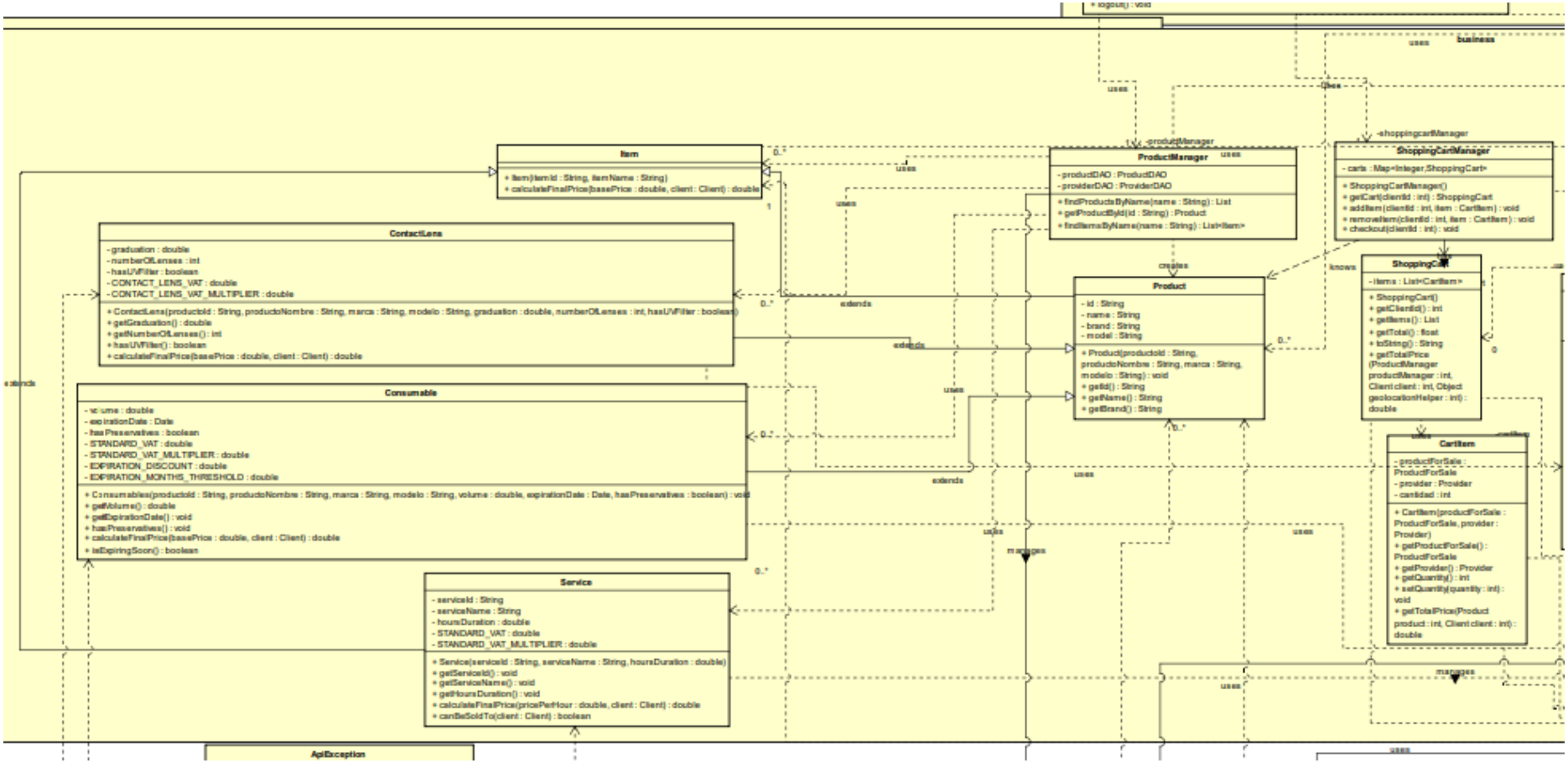
Class Diagram0 2026/01/04



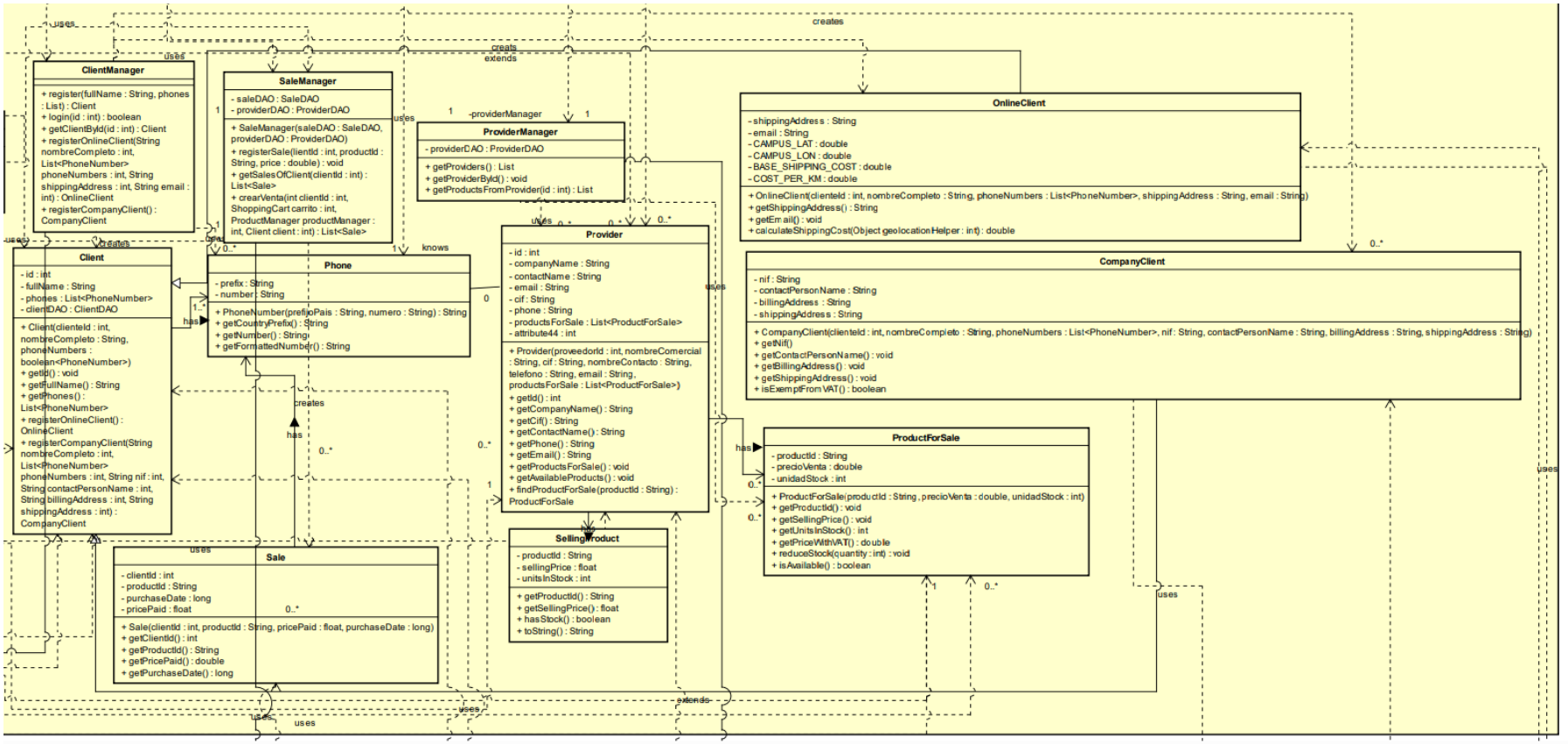
CAPA PRESENTATION



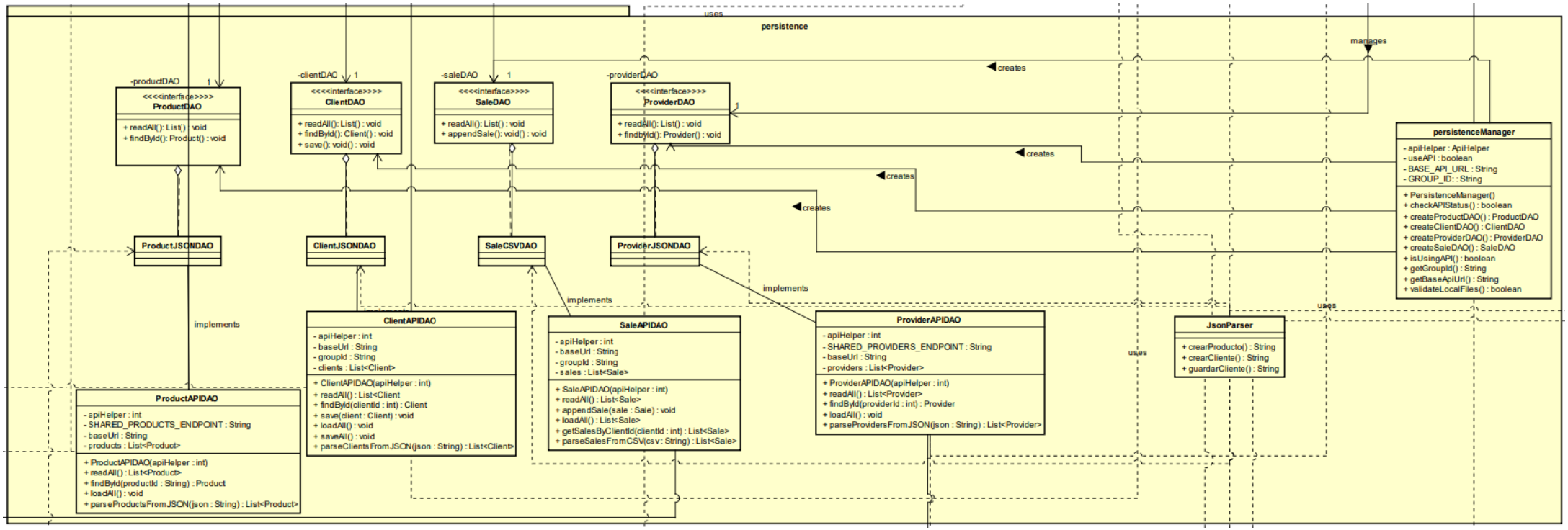
CAPA BUSINESS PARTE 1



CAPA BUSINESS PARTE 2



CAPA PERSISTENCE



El diagrama de clases refleja una evolución significativa respecto a las entregas anteriores. A continuación, explicamos las decisiones estructurales más importantes que justifican el diseño actual:

Abstracción de la Capa de Datos (PersistenceManager)

La decisión más crítica de esta fase ha sido cómo gestionar la dualidad entre la persistencia local (archivos) y la remota (API) sin acoplar la lógica de negocio. Se ha introducido la clase PersistenceManager en la capa de persistencia.

La responsabilidad de esta clase es verificar la disponibilidad de la API al inicio de la ejecución e instanciar dinámicamente las implementaciones DAO correspondientes (ProductAPIDAO o ProductJSONDAO).

Observamos que el Main y el Controller no dependen de clases concretas de acceso a datos, sino que delegan en PersistenceManager. Esto permite cambiar la fuente de datos (de disco a nube) sin tocar una sola línea de los *Managers* de negocio.

Polimorfismo en la Jerarquía de Productos (Item)

Para soportar las nuevas reglas de precios (descuentos por caducidad, IVA reducido, servicios por horas), el diseño anterior basado en una única clase Product era insuficiente. Se ha creado la clase abstracta Item (que extiende de Product) y se han derivado tres subclases especializadas: Consumable, ContactLens y Service.

Se ha aplicado el patrón **Polimorfismo** para el cálculo de precios. En lugar de tener un switch gigante en el Controlador para verificar el tipo de producto, delegamos la lógica en el método abstracto calculateFinalPrice(basePrice, client).

- Consumable: Encapsula la lógica de verificar fechas y aplicar el 40% de descuento.
- ContactLens: Fuerza el IVA del 10% (producto sanitario).
- Service: Calcula el coste en base a horas * tarifa.

La relación de herencia permite que CartItem trate a todos los objetos como Item genéricos, simplificando enormemente el mantenimiento del carrito de compra.

Especialización de Clientes (Online vs Company)

La introducción de costes de envío y exenciones fiscales requería que el sistema distinguiera comportamientos entre tipos de clientes. Se ha extendido la clase base Client hacia OnlineClient y CompanyClient. Siguiendo el principio de **Experto en Información**:

- OnlineClient es el único responsable de saber su dirección y calcular la distancia al almacén (usando el GeolocationHelper) para determinar el coste de envío.
- CompanyClient encapsula la lógica fiscal (isExemptFromVAT), determinando si su dirección de facturación es internacional para aplicar la exención de impuestos.

El Controller y el SaleManager interactúan con la clase padre Client, pero en tiempo de ejecución se ejecutan los comportamientos específicos de las subclases, limpiando la lógica de control.

Desacoplamiento de Managers y DAOs

En el diagrama se observa que cada *Manager* de negocio (ProductManager, SaleManager) tiene una relación de asociación con una **Interfaz DAO** (ProductDAO), nunca con la clase concreta (ProductJSONDAO o ProductAPIDAO).

Esto es fundamental para la testabilidad y flexibilidad del sistema. Permite que el sistema funcione en modo "híbrido" (por ejemplo, leyendo productos de la API pero guardando ventas en CSV local si fuera necesario) de forma transparente.

Gestión Centralizada del Parseo (JsonParser)

A pesar de tener múltiples fuentes de datos (API y Archivos), se ha mantenido una única clase de utilidad JsonParser..Los nuevos DAOs de API (ProductAPIDAO) reutilizan el JsonParser para convertir las respuestas HTTP en objetos Java. Evita la duplicación de lógica de mapeo (DRY). Si la estructura del objeto Product cambia, solo es necesario actualizar el JsonParser, y el cambio se propaga automáticamente tanto a la persistencia local como a la remota.

CONTEXTO

El proyecto Nude Eye nace de la necesidad de digitalizar y mejorar la experiencia de compra de productos ópticos a través de una plataforma centrada en el usuario. Con el objetivo de ser un sistema creado por y para usuarios de gafas, el desarrollo se ha enfocado en simplificar la complejidad técnica que implica gestionar un catálogo de productos de múltiples proveedores, asegurando la integridad de los datos en todo momento.

El objetivo principal ha sido desarrollar un software en Java que no solo cumpla con las funciones básicas de un marketplace, como el registro de usuarios y la gestión de ventas, sino que también implemente arquitecturas avanzadas que se adapten a diferentes entornos de conectividad y al crecimiento del modelo de negocio.

Para hacer realidad esta visión de adaptabilidad, la arquitectura del sistema se ha basado en el patrón Modelo-Vista-Controlador (MVC), garantizando una independencia clara entre la interacción del usuario y la lógica interna. En esta fase final, el sistema ha evolucionado hacia una infraestructura de persistencia híbrida. Gracias a la implementación del patrón DAO (Data Access Object) y una clase gestora, “PersistenceManager”, la aplicación puede operar conectándose a una API remota en tiempo real o utilizando almacenamiento local (JSON/CSV) cuando no hay conectividad, asegurando así la disponibilidad del servicio.

Por otro lado, se ha profundizado en el uso del polimorfismo y la herencia para modelar reglas de negocio complejas sin sacrificar la escalabilidad del código. La especialización de entidades, como OnlineClient para cálculos de envío por geolocalización o la jerarquía de Item para reglas de precios variables, permite que el sistema gestione automáticamente diversas situaciones fiscales y logísticas, manteniendo un eje de negocio limpio, modular y listo para futuras expansiones del catálogo.

ARQUITECTURA DEL SOFTWARE

La solidez del sistema se fundamenta en una arquitectura por capas, diseñada para minimizar las dependencias entre la interfaz de usuario y el núcleo del negocio. Siguiendo este patrón, el software se divide en tres niveles: Presentación, Negocio y Persistencia.

En la capa de presentación, se ha implementado un sistema de controladores jerárquicos donde la clase principal “Controller” coordina la ejecución global y la navegación. Este delega responsabilidades específicas a sub-controladores especializados, como “ControladorBusquedaProducto, ControladorCarrito o ControladorPerfilCliente”. Esta estructura permite que la lógica de interacción con el usuario sea modular, facilitando la corrección de errores y la futura expansión de la interfaz.

En la capa de negocio, el núcleo del sistema se organiza mediante el uso de "Managers", como “ProductManager, ClientManager, SaleManager y ShoppingCartManager”, los cuales actúan bajo el patrón Information Expert. Estas clases son las encargadas de procesar la lógica compleja, como el cálculo de impuestos (IVA), la validación de credenciales y la orquestación de transacciones de venta. Por ejemplo, el “SaleManager” coordina la creación de la venta con la actualización del stock en el proveedor correspondiente, manteniendo las clases de dominio, como “Product” o “Client”, sin problemas de lógica de control y enfocadas únicamente en representar la información del sistema.

Finalmente, la capa de persistencia ha sido diseñada para ofrecer una abstracción total sobre el origen de los datos mediante el patrón DAO (Data Access Object). Se han definido interfaces, como “ProductDAO, ClientDAO”, que permiten al sistema ser agnóstico al formato de almacenamiento. La implementación actual utiliza un “JsonParser” desarrollado a medida para gestionar archivos JSON y CSV de forma local.

DECISIONES DEL DISEÑO

Arquitectura de Persistencia Híbrida

Uno de los retos más difíciles de este proyecto, en concreto de la fase 4, consistía en garantizar que el sistema pudiera operar tanto con datos locales como remotos sin alterar la lógica de negocio preexistente. Para ello, se ha implementado la clase “PersistenceManager”, la cual actúa como un gestor encargado de determinar la fuente de datos durante el tiempo de ejecución. Durante la inicialización, el sistema verifica la disponibilidad de la API. Si la conexión es exitosa, se procede a la creación e inyección de los objetos “ProductAPIDAO” y “ClientAPIDAO”. En caso de detectarse una desconexión, se activa un mecanismo de *fallback* que redirige el flujo hacia “ProductJSONDAO” y “ClientJSONDAO”. Esta estrategia permite que los gestores de negocio dependan exclusivamente de interfaces y no del origen físico de la información, lo que simplifica significativamente el mantenimiento y la escalabilidad de la infraestructura.

Polimorfismo en el Modelo de Dominio

En algunas fases, el cálculo del precio final dependía de múltiples estructuras condicionales para identificar el tipo de producto, elevando la dificultad del código. Pero para la cuarta, se ha reestructurado la clase base “Product” mediante la introducción de la clase abstracta “Item”, que define el contrato “calculateFinalPrice”. Gracias a esta jerarquía, la clase “Consumable” gestiona automáticamente los descuentos por caducidad, “ContactLens” aplica el IVA reducido del 10% por su finalidad sanitaria y “Service” parametriza el coste según las horas de dedicación. Este uso del polimorfismo reduce la complejidad del flujo en el “ShoppingCart” y permite la futura expansión del catálogo sin necesidad de modificar el código fuente existente.

Especialización de Clientes y Lógica Fiscal

La gestión de envíos y las exenciones fiscales exigían comportamientos específicos que, para mantener una arquitectura limpia, no debían separarse en la capa de control. Se ha decidido optar por una especialización de las entidades. “OnlineClient” incorpora, para la fase 4, la lógica necesaria para calcular costes de envío dinámicos mediante la fórmula de Haversine, determinando la distancia geográfica respecto al Campus La Salle. Por otro lado, “CompanyClient” asume la responsabilidad de validar las exenciones de IVA a través del método “isExemptFromVAT”. Al delegar estas decisiones en los propios objetos que poseen

la información, se evita saturar la lógica condicional en los controladores y garantizar así un eje de negocio más cohesivo y modular.

Gestión de Datos y Serialización Centralizada

Toda la lógica de transformación entre cadenas de texto y objetos se ha unificado en la clase “JsonParser”, que presta servicio tanto a los DAOs de archivos locales como a las respuestas obtenidas de la API remota. Esta decisión evita la duplicación de código de parseo en diferentes niveles de la persistencia. De esta manera, cualquier ajuste en la estructura del formato de datos se resuelve en un único punto del sistema, asegurando la consistencia y la integridad de la información en todo momento.

GESTIÓN DE DATOS

La capa de persistencia de este proyecto no se limita a solo la lectura y escritura de archivos, sino que implementa una estrategia de resiliencia de datos diseñada para entornos difíciles. El sistema garantiza la integridad y disponibilidad de la información mediante una arquitectura compleja que previene la pérdida de datos ante fallos externos.

En primer lugar, se ha implementado un mecanismo de conmutación por error. Durante el arranque, el software realiza un diagnóstico del estado de una API externa. Si esta conexión falla, el sistema no se detiene; en su lugar, redirige dinámicamente el flujo de datos hacia una infraestructura local basada en un “JsonParser”. Este componente es capaz de reconstruir estructuras complejas de objetos sin dependencias externas, transformando archivos .json y .csv en instancias de negocio en su funcionabilidad total.

La elección de formatos es estratégica, ya que se utiliza JSON para entidades con relaciones anidadas y jerárquicas (como productos y proveedores con sus listas de stock) y CSV para la persistencia de ventas mediante el “SaleCSVDAO”. Es en este último donde se aplica una técnica de escritura basada en la adición consecutiva de registros. A diferencia de los métodos de sobrescritura, este modelo asegura que cada nueva transacción se anexe al final del archivo sin alterar ni procesar los datos preexistentes.

Esta decisión de diseño es fundamental para la seguridad del sistema por tres motivos principales. El primero, establece un historial de ventas inmutable, donde la información antigua queda protegida contra borrados accidentales o corrupciones durante el guardado. El segundo, maximiza la eficiencia en operaciones de registro masivo, ya que el software no necesita cargar en memoria miles de ventas previas para registrar una nueva. Y el tercero, garantiza la integridad transaccional, puesto que, si el proceso se interrumpe bruscamente, el resto del archivo histórico permanece intacto. Gracias al patrón DAO, toda esta complejidad técnica queda oculta para la capa de negocio, permitiendo que los "Managers" operen de forma independiente al origen físico de la información y al método específico de almacenamiento empleado.

DOMINIO Y JERARQUIA DE ENTIDADES

El modelado del dominio de este proyecto se ha estructurado bajo la premisa de la coherencia lógica y el modularidad, capturando la complejidad técnica del sector óptico mediante una jerarquía de clases especializadas. En lugar de utilizar estructuras de datos planas, se ha diseñado un grafo de objetos que permite una representación precisa de la realidad del negocio, asegurando que cada entidad cumpla con su propósito sin solapamientos de responsabilidad.

Un elemento clave en esta arquitectura es la clase “ProductForSale”, que actúa como una entidad de mediación entre el producto genérico y el proveedor específico. Esta clase no se limita a ser un contenedor de datos, sino que además incorpora lógica de negocio propia como el cálculo dinámico de impuestos (aplicando un IVA al 21%) y el control de disponibilidad en tiempo real mediante el método “isAvailable()”. Asimismo, la entidad “Client” implementa una relación de composición con objetos “PhoneNumber”, lo que permite al sistema normalizar prefijos internacionales y gestionar múltiples líneas de contacto de forma estructurada.

La gestión de la temporalidad también ha sido crítica en el diseño. La clase “Sale” utiliza tipos primitivos de alta capacidad (long) para almacenar *timestamps*. Esta decisión técnica permite que el sistema realice operaciones de orden y comparación cronológica con una carga computacional mínima, facilitando que el “SaleManager” procese historiales de compra extensos o genere estadísticas de consumo sin comprometer el rendimiento de la aplicación ni la experiencia del usuario final.

FLUJOS DE CONTROL

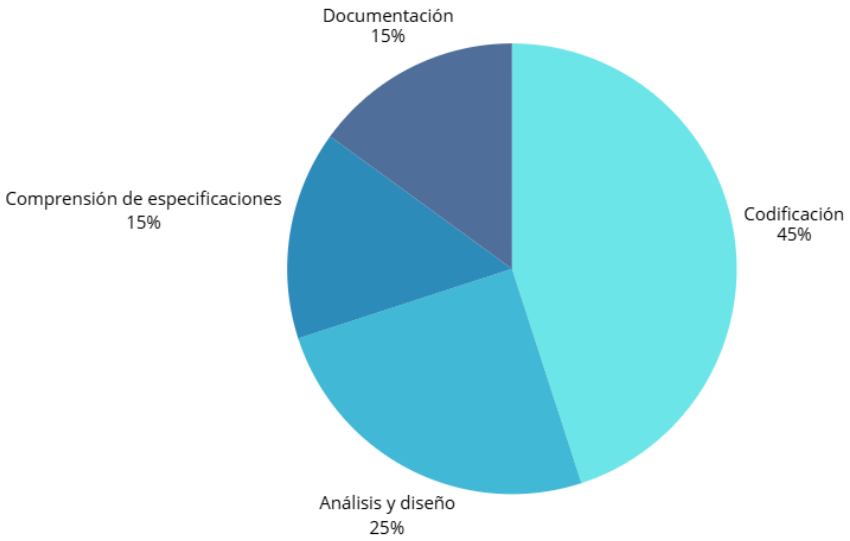
La integridad del sistema y la protección de la información se han asegurado mediante una arquitectura de defensa en profundidad, interceptando las acciones del usuario en múltiples niveles de control antes de que cualquier dato impacte en la persistencia. El flujo se inicia con un mecanismo de autenticación complejo gestionado por el “ControladorLogin”, que actúa como guardián de acceso a las funcionalidades sensibles de la plataforma.

La validación de datos se ejecuta de forma transversal y se divide en dos fases que garantizan la salud del sistema:

1. Validación Sintáctica: En la capa de presentación, a través de la clase “MainUI”, se emplean métodos de captura segura de datos que utilizan expresiones regulares para validar formatos, como los prefijos telefónicos. Esto asegura que solo información con la estructura correcta sea transmitida hacia el interior del sistema, actuando como un primer filtro contra datos erróneos.
2. Validación Semántica: Posteriormente, en la capa de negocio, los “Managers” aplican la validación lógica. Por ejemplo, el “ShoppingCartManager” valida la disponibilidad de stock real antes de permitir cualquier modificación en el carrito, mientras que el “ClientManager” coordina la generación de identificadores únicos para evitar colisiones de identidad en los registros locales.

Esta separación de responsabilidades garantiza que la capa de presentación nunca manipule directamente las entidades de dominio sin la supervisión de un controlador, cumpliendo con el principio de mínimo privilegio y asegurando que cada transacción sea coherente y segura ante entradas inesperadas.

DEDICACIÓN



- Codificación: 45%

Esta ha sido la parte más exigente. La implementación de la comunicación HTTP manual a través de ApiHelper y la gestión de los códigos de respuesta consumió una gran parte del tiempo. Además, la reestructuración para introducir la jerarquía de Item implicó modificar la lógica de creación en todo el proyecto y adaptar los tests manuales.

- Análisis y Diseño: 25%

Antes de comenzar a programar, fue necesario rediseñar el diagrama UML en cada fase, para acomodar las nuevas jerarquías y definir cómo, por ejemplo el PersistenceManager interactuaría con el Main", evitando así dependencias circulares y asegurando una arquitectura limpia.

- Comprensión de especificaciones: 15%

Se necesitó comprender algunos conceptos que no estaban claros del todo como la documentación de la API docente y de las reglas de negocio fiscales (IVA intracomunitario) antes de poder proponer una solución técnica viable.

- Documentación: 15%

La redacción de Javadoc estandarizado para las nuevas clases y la elaboración de las memorias.

CONCLUSIONES

Este proyecto demuestra que la complejidad de una aplicación no reside únicamente en su capacidad funcional, sino en la solidez de una arquitectura diseñada para afrontar escenarios de error y crecimiento. A lo largo del desarrollo de Nude Eye Project, se ha priorizado la creación de un ecosistema donde la lógica de negocio permanece totalmente independiente de las capas externas, garantizando que cualquier evolución en el soporte físico o en la interfaz no comprometa la estabilidad del núcleo.

Uno de los logros técnicos alcanzados ha sido la implementación de una infraestructura de persistencia resiliente. La capacidad del software para derivar entre fuentes de datos de forma transparente asegura que la disponibilidad del servicio nunca se vea comprometida, incluso ante fallos de conectividad externos. Asimismo, la rigurosidad en los flujos de control garantiza que cualquier proceso atípico sea interrumpido o invalidado antes de que pueda impactar en los registros, protegiendo así la integridad histórica de la plataforma.

Finalmente, el uso estratégico de la herencia y el polimorfismo ha permitido alcanzar un nivel de especificidad en el modelado de datos que facilita el mantenimiento y la extensibilidad a largo plazo. Al evitar el acoplamiento y apostar por las dependencias, el sistema no solo cumple con los requisitos actuales del sector óptico, sino que se posiciona como una base técnica preparada para escalar hacia entornos de producción más complejos sin comprometer su rendimiento ni su seguridad estructural.

BIBLIOGRAFÍA

La Salle–Universitat Ramon Llull. (n.d.). *eStudy 2526: Folder view (id=88896)* [Carpeta de curso; requiere inicio de sesión]. https://estudy2526.salle.url.edu/mod/folder/view.php?id=88896

Oracle. (n.d.). *Java language basics*. Dev.java. https://dev.java/learn/language-basics/ (dev.java)

Smart Programming. (n.d.). *Dependency (USES-A relationship) in Java*. https://smartprogramming.in/tutorials/java/dependency-uses-a-relationship-in-java (smartprogramming.in)

Roca, D. (2022, August 26). *Uso de APIs en Java*. Escola Espai. https://www.espai.es/blog/2022/08/apis-en-java/ (Espai)

LogicalDOC. (n.d.). *API Java*. LogicalDOC Documentation. https://docs.logicaldoc.com/es/java-api (LogicalDOC Documentation)

Blancarte, O. (2018, December 10). *Data Access Object (DAO) pattern*. Oscar Blancarte – Software Architecture. https://www.oscarblancarteblog.com/2018/12/10/data-access-object-dao-pattern/ (Oscar Blancarte - Software Architecture)

Baeldung. (2025, March 26). *The DAO pattern in Java*. https://www.baeldung.com/java-dao-pattern (baeldung.com)